PLANO DE AULA + APOSTILA

Aula 1 — PDO Avançado e Relacionamentos 1:N

Desenvolvimento de Sistemas Web • Nível Intermediário

Pré-requisito: CRUD e Autenticação com Sessions em PHP

Plano de Aula

Aula 1 — PDO Avançado e Relacionamentos 1:N | Duração: 50 minutos	
Objetivo	Compreender o conceito de relacionamento entre tabelas e aplicar JOINs com PDO para exibir dados de múltiplas tabelas em uma única consulta.
Conteúdo	• O problema do dado duplicado — por que separar dados em tabelas? (8 min) • Chave primária, chave estrangeira e o relacionamento 1:N (10 min) • O comando JOIN: INNER, LEFT — diferenças na prática (10 min) • Live coding: listagem de contatos com seus telefones via JOIN (17 min) • Exercício prático e revisão (5 min)
Atividade	Alunos criam as tabelas contatos e telefones no banco, inserem dados de teste e constroem uma página PHP que exibe cada contato com todos os seus telefones usando JOIN.

Projeto da aula

Sistema de Agenda de Contatos com múltiplos telefones por contato.

Cada contato pode ter 0 ou mais telefones cadastrados — relacionamento 1:N.

Ao final da aula, a página exibe: nome, e-mail e todos os telefones de cada contato em uma única consulta SQL.

Teoria — Relacionamentos entre Tabelas

1. Por que separar dados em tabelas diferentes?

Imagine que você quer guardar um contato com vários telefones. A solução ingênua seria criar colunas tel1, tel2, tel3 na tabela de contatos. Isso gera vários problemas:

- **Problema 1:** E se o contato tiver 4 telefones? Você teria que alterar a estrutura do banco.
- **Problema 2:** Se o contato não tiver telefone, as colunas ficam vazias — desperdício.
- **Problema 3:** Impossível buscar 'todos os contatos com telefone de DDD 11' sem gambiarras.

Solução: normalização

Separar as informações em tabelas diferentes, conectadas por chaves estrangeiras.
Tabela contatos → guarda nome e e-mail.
Tabela telefones → guarda cada telefone COM uma referência ao contato dono dele.
Assim um contato pode ter quantos telefones quiser, sem alterar a estrutura.

2. Chave Primária e Chave Estrangeira

Termo	Definição
`PRIMARY KEY`	Identificador único de cada linha. Nunca se repete, nunca é nulo. Geralmente é um INT AUTO_INCREMENT.
`FOREIGN KEY`	Coluna que referencia a PRIMARY KEY de outra tabela. Cria o elo entre as tabelas. Também chamada de FK.
`Relacionamento 1:N`	Um registro de uma tabela se relaciona com vários da outra. Ex: 1 contato → N telefones.
`Integridade referencial`	Regra que impede inserir um telefone com um contato_id que não existe na tabela contatos.
`CASCADE`	Comportamento opcional: ao deletar um contato, o banco automaticamente deleta todos os telefones dele.

3. Estrutura das Tabelas

SQL

```sql
-- Tabela principal: um contato
CREATE TABLE contatos (
    id          INT          AUTO_INCREMENT PRIMARY KEY,
    nome        VARCHAR(100) NOT NULL,
    email       VARCHAR(150) NOT NULL UNIQUE,
    criado_em   TIMESTAMP    DEFAULT CURRENT_TIMESTAMP
);
```

```
-- Tabela dependente: cada linha é UM telefone de UM contato
CREATE TABLE telefones (
    id          INT         AUTO_INCREMENT PRIMARY KEY,
    contato_id  INT         NOT NULL,          -- chave estrangeira
    tipo        VARCHAR(20) NOT NULL, -- "celular", "fixo", "trabalho"
    numero      VARCHAR(20) NOT NULL,
     -- Cria o elo: contato_id referencia id da tabela contatos
    FOREIGN KEY (contato_id)
        REFERENCES contatos(id)
        ON DELETE CASCADE   -- se o contato for deletado, telefones
somem também
);
```

Visualizando o relacionamento

contatos telefones

id	nome	email	id	contato_id	tipo	numero
1	Maria	m@m.com	1	1	celular	11 99999-0001
2	João	j@j.com	2	1	fixo	11 3333-0002
			3	2	celular	21 98888-0003

Maria (id=1) tem 2 telefones. João (id=2) tem 1 telefone.

4. O comando JOIN

JOIN é a instrução SQL que une duas tabelas em uma única consulta. Em vez de fazer duas queries separadas, você busca tudo de uma vez.

INNER JOIN — só retorna quando existe correspondência nos dois lados

```
SQL
-- Retorna contatos que TÊM pelo menos 1 telefone
SELECT
    c.id,
    c.nome,
    c.email,
    t.tipo,
    t.numero
FROM contatos c
INNER JOIN telefones t ON t.contato_id = c.id
ORDER BY c.nome, t.tipo;

-- Se Maria tem 2 telefones, ela aparece 2 vezes na listagem.
-- Se um contato não tem telefone, ele NÃO aparece (isso é INNER).
```

LEFT JOIN — retorna todos da esquerda, mesmo sem correspondência

```sql
-- Retorna TODOS os contatos, com ou sem telefone
SELECT
    c.id,
    c.nome,
    c.email,
    t.tipo,
    t.numero
FROM contatos c
LEFT JOIN telefones t ON t.contato_id = c.id
ORDER BY c.nome;

-- Se um contato não tem telefone, t.tipo e t.numero ficam NULL.
-- Use LEFT JOIN quando precisa listar todos, inclusive os sem relação.
```

INNER vs LEFT — quando usar cada um?

INNER JOIN → use quando a relação é obrigatória para o resultado.
Exemplo: "Liste todos os pedidos com seus produtos" (pedido sem produto é inválido).

LEFT JOIN → use quando a relação é opcional.
Exemplo: "Liste todos os contatos, mostrando telefones se houver" (contato sem telefone é válido).

PDO na Prática — Buscando Dados Relacionados

5. Estratégias para exibir dados 1:N em PHP

Quando um contato tem vários telefones, a query JOIN retorna uma linha por telefone. Precisamos de uma estratégia para agrupar esses resultados em PHP.

Estratégia A — GROUP_CONCAT (simples, boa para listagens)

O MySQL agrupa todos os telefones em uma única string separada por vírgula. Mais simples, mas perde a estrutura individual de cada telefone.

```sql
SELECT
    c.id,
    c.nome,
    c.email,
    GROUP_CONCAT(t.numero ORDER BY t.tipo SEPARATOR " | ") AS telefones
FROM contatos c
LEFT JOIN telefones t ON t.contato_id = c.id
GROUP BY c.id, c.nome, c.email
ORDER BY c.nome;

-- Resultado:
-- id | nome  | email     | telefones
-- 1  | Maria | m@m.com   | 11 99999-0001 | 11 3333-0002
-- 2  | João  | j@j.com   | 21 98888-0003
```

Estratégia B — Agrupamento em PHP (mais flexível, recomendada)

Fazemos o JOIN simples e agrupamos os resultados em PHP, montando um array associativo. Mais controle sobre cada telefone individualmente.

```php
<?php
function listarContatosComTelefones(PDO $pdo): array
{
    $sql = "
        SELECT
            c.id          AS contato_id,
            c.nome,
            c.email,
            t.id          AS telefone_id,
            t.tipo,
            t.numero
        FROM contatos c
        LEFT JOIN telefones t ON t.contato_id = c.id
        ORDER BY c.nome, t.tipo
    ";

    $stmt = $pdo->query($sql);
    $rows  = $stmt->fetchAll();

    // Agrupar: um array por contato, com sub-array de telefones
    $contatos = [];
    foreach ($rows as $row) {
        $cid = $row["contato_id"];
```

```
        // Cria a entrada do contato na primeira vez que o vemos
        if (!isset($contatos[$cid])) {
            $contatos[$cid] = [
                "id"        => $cid,
                "nome"      => $row["nome"],
                "email"     => $row["email"],
                "telefones" => [],          // começa vazio
            ];
        }

        // Adiciona o telefone se existir (LEFT JOIN pode retornar NULL)
        if ($row["telefone_id"] !== null) {
            $contatos[$cid]["telefones"][] = [
                "id"     => $row["telefone_id"],
                "tipo"   => $row["tipo"],
                "numero" => $row["numero"],
            ];
        }
    }

    return array_values($contatos); // reindexar para array simples
}
```

Por que array_values() no final?

O array $contatos usa o id do contato como chave (ex: [1 => ..., 2 => ...]).
array_values() transforma para índices 0, 1, 2... — mais fácil de iterar no foreach.
Sem isso, ao percorrer com foreach no HTML, as chaves numéricas podem causar confusão.

6. Exibindo no HTML com PHP

```PHP
<?php
require_once "config/database.php";

$pdo      = getConexao();
$contatos = listarContatosComTelefones($pdo);
?>
<!DOCTYPE html>
<html lang="pt-BR">
<head>
    <meta charset="UTF-8">
    <title>Agenda de Contatos</title>
    <link rel="stylesheet" href="assets/style.css">
</head>
<body>
<div class="container">
    <h1>Agenda de Contatos</h1>

    <?php foreach ($contatos as $contato): ?>
    <div class="card-contato">

        <div class="contato-header">
```

```
            <strong><?= htmlspecialchars($contato["nome"]) ?></strong>
            <span class="email"><?= htmlspecialchars($contato["email"])
?></span>
        </div>

        <div class="telefones">
            <?php if (empty($contato["telefones"])): ?>
                <span class="sem-tel">Nenhum telefone cadastrado</span>

            <?php else: ?>
                <?php foreach ($contato["telefones"] as $tel): ?>
                <span class="telefone">
                    <span class="tipo"><?=
htmlspecialchars($tel["tipo"]) ?></span>
                    <?= htmlspecialchars($tel["numero"]) ?>
                </span>
                <?php endforeach; ?>
            <?php endif; ?>
        </div>

    </div>
    <?php endforeach; ?>

</div>
</body>
</html>
```

Extra — Paginação com LIMIT e OFFSET

Quando a tabela tem muitos registros, exibir todos de uma vez é lento. Paginação divide os resultados em páginas usando LIMIT e OFFSET.

Termo	Definição
`LIMIT n`	Retorna no máximo n linhas. Ex: LIMIT 10 retorna as primeiras 10 linhas.
`OFFSET n`	Pula os primeiros n resultados. Ex: OFFSET 10 começa da linha 11.
`ceil()`	Função PHP que arredonda para cima. Usada para calcular o total de páginas.
`$_GET['pagina']`	Parâmetro passado na URL. Ex: listagem.php?pagina=2 vai para a página 2.

PHP

```php
<?php
$porPagina  = 10;                              // itens por página
$paginaAtual = max(1, (int)($_GET["pagina"] ?? 1)); // mínimo 1
$offset     = ($paginaAtual - 1) * $porPagina; // quantos pular

// 1. Contar o total de contatos
$total = $pdo->query("SELECT COUNT(*) FROM contatos")->fetchColumn();
$totalPaginas = (int) ceil($total / $porPagina);

// 2. Buscar só os contatos desta página
$stmt = $pdo->prepare(
    "SELECT id, nome, email FROM contatos
     ORDER BY nome
     LIMIT :limite OFFSET :offset"
);
$stmt->bindValue(":limite",  $porPagina,  PDO::PARAM_INT);
$stmt->bindValue(":offset",  $offset,     PDO::PARAM_INT);
$stmt->execute();
$contatos = $stmt->fetchAll();

// Dica: LIMIT e OFFSET exigem bindValue com PARAM_INT.
// Passar como array no execute() pode falhar com alguns drivers.
```

Fórmula da paginação

Página 1: LIMIT 10 OFFSET 0 → linhas 1 a 10
Página 2: LIMIT 10 OFFSET 10 → linhas 11 a 20
Página 3: LIMIT 10 OFFSET 20 → linhas 21 a 30

Fórmula: offset = (pagina_atual - 1) × itens_por_pagina

Glossário da Aula

Termo	Definição
`JOIN`	Cláusula SQL que une duas ou mais tabelas em uma única consulta com base em uma condição.
`INNER JOIN`	Retorna apenas as linhas que têm correspondência em ambas as tabelas.
`LEFT JOIN`	Retorna todas as linhas da tabela da esquerda, mesmo sem correspondência na direita (NULL).
`ON`	Condição do JOIN. Define qual coluna de uma tabela corresponde a qual da outra.
`PRIMARY KEY`	Identificador único de cada linha de uma tabela. Não pode ser nulo nem repetido.
`FOREIGN KEY`	Coluna que referencia a PRIMARY KEY de outra tabela. Cria o relacionamento.
`ON DELETE CASCADE`	Ao deletar o registro pai, todos os filhos relacionados são deletados automaticamente.
`GROUP_CONCAT()`	Função MySQL que concatena valores de um grupo em uma única string.
`GROUP BY`	Agrupa linhas com o mesmo valor em colunas especificadas. Usado com funções de agregação.
`LIMIT`	Define o número máximo de linhas retornadas pela query.
`OFFSET`	Define quantas linhas pular antes de começar a retornar resultados.
`fetchAll()`	Método PDO que retorna todas as linhas do resultado como array de arrays associativos.
`fetchColumn()`	Retorna o valor da primeira coluna da primeira linha. Útil para COUNT(*).
`bindValue()`	Liga um valor a um parâmetro nomeado do prepared statement, com tipo explícito.
`PDO::PARAM_INT`	Constante PDO que define o tipo inteiro para bindValue(). Obrigatório para LIMIT e OFFSET.
`array_values()`	Reindexação de array em PHP: transforma chaves arbitrárias em 0, 1, 2...
`htmlspecialchars()`	Escapa caracteres HTML para prevenir XSS. Usar em TODA saída de dados do banco.

Exercício da Aula

Tarefa

1. Crie o banco agenda_db e as tabelas contatos e telefones conforme o SQL da aula.

2. Insira ao menos 3 contatos, com 2 ou mais telefones cada.
3. Crie a página listagem.php que exibe todos os contatos com seus telefones agrupados.
4. Desafio: adicione um formulário simples de cadastro de novo contato.
5. Desafio extra: adicione paginação com 2 itens por página para testar.

Dica para o professor

Mostre ao vivo no phpMyAdmin o resultado bruto do SELECT JOIN antes de agrupar em PHP.

Os alunos precisam ver que Maria aparece 2 vezes na query e que o PHP 'junta' isso.

Esse momento de 'ah, entendi!' costuma ocorrer quando eles comparam a query com o array final.